FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

SYSTEM METHODOLOGY REPORT

Web-Based Scheduling & Final Year Project Supervision System

A comprehensive overview of the architectural framework, operational methodologies, and implementation stack engineered for academic research supervision.

Iterative Agile Implementation & Technical System Specifications

---

Core Technologies: React 19 • Node.js • Express • Agora RTC (WebRTC) • MongoDB / JSON DB

Generated on: June 30, 2026

1.0 Introduction to the System Methodology

The administration of final year undergraduate and postgraduate academic projects has traditionally suffered from procedural bottlenecks, fragmented communication channels, uncoordinated document revisions, and scheduling conflicts. To address these systemic inefficiencies, this system adopts a rigorous, web-based engineering methodology aimed at unifying the entire supervision lifecycle into a single interactive platform.

The core methodology merges user-centered interface designs with stateful backend control flows. It constructs a secure collaborative workspace for three distinct user roles:

- **Students:** Empowered to propose research directions, upload chapter drafts, book office hours, and present project milestones in real-time.
- **Supervisors:** Provided with custom dashboard panels to oversee multiple assigned students, offer asynchronous feedback on specific documents, manage appointment calendars, and host defense presentations.
- **Administrators:** Enabled to coordinate user profiles, enforce supervisor assignments, monitor overall milestones, and maintain audit logs.

2.0 System Development Life Cycle (SDLC)

The project implementation follows the **Agile Development Methodology** utilizing iterative sprints. Agile development is chosen to facilitate rapid prototyping, continuous stakeholder integration, and the ability to refine capabilities based on user feedback during the academic semester.

> **Iterative Feedback Cycle:** Unlike the traditional sequential Waterfall model, the Agile framework ensures that supervisors and students can review partial releases (e.g. initial topic proposal pipelines) while scheduling and video features are undergoing design. This eliminates the risk of building incorrect workflows.

2.1 Sprint Breakdown

Sprint Cycle	Key Deliverables	Verification Method
Sprint 1: Core & RBAC	Vite development environment, Express REST API, Bcrypt hashing, and JSON Web Token (JWT) Authentication.	Integration tests for route protection and payload authentication tokens.
Sprint 2: Topic & Document Workflow	Topic proposal pipeline, chapter file upload handler via Multer, metadata tagging, and review states.	State machine validation tests (e.g., Pending → Approved/Revision).
Sprint 3: Scheduling & Real-time Video	Interactive booking calendar, venue/link mapping, and WebRTC integration via the Agora RTC SDK.	P2P communication tests and microphone/camera diagnostic suites.
Sprint 4: Alerts & Deployment	In-app notifications, Nodemailer SMTP service integration, production bundling, and MongoDB bridging.	SMTP delivery verification and type checking.

3.0 Requirements Engineering & Specifications

Detailed requirement engineering ensures the system provides correct access control boundaries, low latency operations, and dependable real-time communications.

3.1 Functional Requirements

- **Authentication & Role Enforcement:** Secure registration and login. Session persistence via HTTP headers containing JWTs. Automatic routing based on user role (student, supervisor, admin).
- **Topic Proposal State Machine:** Students submit proposals. Supervisors are alerted and can transition the status to `approved`, `revision`, or `rejected` with text-based feedback.
- **Milestone Document Repository:** Document uploads (up to 10MB) categorized with specific tags (e.g., "Chapter 1", "Chapter 2", "Presentation Slides"). Supports version uploads and inline supervisor feedback strings.
- **Meeting Scheduler:** Dynamic booking interface where students request meetings by specifying titles, dates, times, and channels (physical venue or virtual link). Supervisors accept or reschedule.
- **Integrated Virtual Defense:** An embedded virtual conference room utilizing WebRTC to facilitate remote supervisor-student consultations, slide reviews, and project defenses directly in the web browser.

3.2 Non-Functional Requirements

- **Performance & Response Time:** Server API endpoints respond within 150ms on average under standard load. Client-side navigation is instant using React Single-Page Application (SPA) architecture.
- **Information Security & Encryption:** All sensitive credentials (passwords) are irreversibly hashed using BcryptJS with 10 salt rounds. Communication is encrypted via TLS/HTTPS.
- **High Availability & Resilience:** Graceful fallbacks for WebRTC/Agora connections, local file database backups, and persistent email logging to track communication failures.

4.0 System Architecture & Structural Design

The application is engineered as a **Single-Port Client-Server Web Application**. Using Vite's development proxy and unified production bundles, both the Express API endpoints and Vite's frontend assets are served from a single port, resolving CORS configuration issues and streamlining cloud deployments.

4.1 Layered Architecture

The system follows a classic three-tier architecture model to separate concerns, maintain clean code, and enable modular testing:

1. Presentation Layer (Client Frontend)

Built with **React 19**, **TypeScript**, and **Tailwind CSS v4**. Utilizes client-side routing, responsive dashboard layouts, context-based state containers (`AuthContext`, `SupervisionContext`), and dynamic user interactions facilitated by Framer Motion and Lucide icons.

2. Service & Business Logic Layer (Server API)

Implemented with **Node.js** and **Express**. This layer receives incoming HTTP requests, intercepts unauthorized traffic using custom JWT Verification Middleware, processes uploads via Multer, forwards notifications, and runs validations.

3. Data Access & Storage Layer

Features a dual-mode persistence architecture:

- **Development Sandbox Mode:** A low-overhead, transactional JSON file-store (`data/db.json`) that automatically reads and writes updates atomically on server starts. This allows the system to run out-of-the-box in local preview environments.
- **Production Mode:** Connects to a cloud-hosted **MongoDB Atlas** database cluster using Mongoose schemas. This ensures high availability, scalability, and automatic index optimization.

5.0 Database Schema Design

To guarantee transactional integrity, the database model defines clear associations (e.g. connecting proposals to students, and students to supervisors). The primary entities are outlined below:

User Schema

Field	Type	Description
id	String (UUID / ObjectId)	Primary key. Unique identifier.
email	String	Unique email address used for login and notifications.
password	String	Cryptographically hashed password (Bcrypt).
role	String (Enum)	Role constraints: `student` | `supervisor` | `admin`.
matricNumber	String (Optional)	Student identification number.
supervisorId	String (Foreign Key)	Reference to the assigned supervisor's user ID.

Topic & Proposal Schema

Field	Type	Description
id	String	Unique proposal identifier.
title	String	Proposed research topic title.
description	String	Abstract, problem statement, and research goals.
status	String (Enum)	States: `pending` | `approved` | `rejected` | `revision`.
fileUrl	String (Optional)	Path to the uploaded PDF proposal document.

Schedule Schema

Field	Type	Description
meetingDate	String (Date format)	Scheduled day of the meeting.
time	String	Time slot (e.g., "14:00").
venue	String	Physical office location or WebRTC virtual meeting URL.
status	String (Enum)	States: `pending` | `approved` | `completed` | `cancelled`.

6.0 Verification & Quality Assurance

To ensure the system's reliability and compliance with academic institutional standards, a comprehensive Quality Assurance (QA) protocol was executed across the codebase.

6.1 Security and Authorization Testing

Tests verify that security boundaries are strictly enforced. All protected routes require a valid JSON Web Token:

- Verified that requests without headers receive an HTTP 401 Unauthorized status.
- Verified that student tokens attempting to access supervisor routes (such as updating grades or proposal approvals) trigger an HTTP 403 Forbidden status.
- Verified that administrators are the only roles allowed to modify student-supervisor matching schemas.

6.2 Responsive Design Validation

The web interface was tested across multiple screen viewports to ensure seamless usability for students accessing dashboards from campus mobile networks or supervisors using desktop workstations:

- **Mobile (375px - 480px):** Navigation collapses into a touch-friendly sidebar. Cards stacked vertically.
- **Tablet (768px - 1024px):** Sidebar navigation toggles responsively. Data grids collapse into scrollable tables.
- **Desktop (1200px+):** Full split-screen views displaying supervision details, shared file lists, and calendar slots concurrently.

6.3 Real-Time Communications Diagnostics

WebRTC channels powered by the Agora RTC SDK NG were tested for high latency fallbacks. The system gracefully switches to audio-only transmission if packet loss exceeds 25%, maintaining session continuity.

End of Methodology Report